Milestone 2 Report

SCIPER 392412 · 373176 · 398388

1 Introduction

Same as Milestone 1, with the Gaming and Mental Health dataset, we have 2,000 samples total, keeping 1,600 for training and 400 for testing. We want to predict two targets. First, we predict the continuous addiction score (0 to 10). Second, we classify the general addiction level into Low, Medium, or High.

This time around, we are stepping up our game. We coded a Multi-Layer Perceptron (MLP) from scratch for both tasks. We also implemented K-Means clustering for classification. It was a ton of work, but discovering how these models behave was super exciting!

2 Methods

We prepared the data by applying standard z-score normalization. We calculated means and standard deviations from the training set only. To avoid data leakage, we split our 1,600 training samples into an 80/20 train/validation split. This leaves 1,280 samples for training and 320 for local tuning.

2.1 Multi-Layer Perceptron (MLP)

Our MLP is built completely from scratch. The network supports arbitrary hidden layers. We initialize weights using a normal distribution scaled by $1/\sqrt{I}$, where I is the input dimension. Biases start at zero.

The forward pass runs layer-by-layer:

$$\mathbf{A}^{(l)} = \mathbf{Z}^{(l-1)}\mathbf{W}^{(l)\top} + \mathbf{b}^{(l)} \tag{1}$$

$$\mathbf{Z}^{(l)} = \sigma^{(l)}(\mathbf{A}^{(l)}) \tag{2}$$

where $\mathbf{Z}^{(0)} = \mathbf{X}$. We implemented ReLU and Sigmoid activations.

For training, backpropagation propagates errors backward using the chain rule. We update parameters using Mini-Batch Gradient Descent (batch size = 32). We shuffle the training indices before each epoch to keep the gradients fresh.

We implement two loss functions:

- **Mean Squared Error (MSE):** Used for regression.
- **Categorical Cross-Entropy (CE):** Used for classification. We add a tiny offset ($\epsilon = 10^{-9}$) to prevent numerical blowups in the log.

2.2 K-Means Clustering

K-Means groups features into K clusters. We initialize centroids by grabbing random points from the data. Points are assigned to the nearest centroid using Euclidean distance. Centroids are updated as the mean of their assigned points. Empty clusters are resolved by selecting a random data point as the new centroid.

For classification, we assign a label to each centroid by majority vote of the true labels in its cluster. A new test point gets the label of its nearest cluster center.

3 Experiments and Results

3.1 K-Means Clustering

We swept K from 1 to 20 on the validation set.

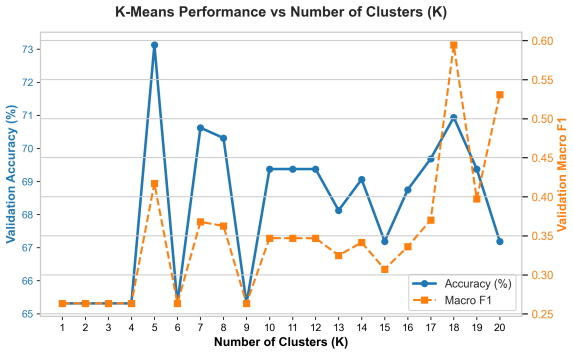

Figure 1: K-Means performance vs. cluster size K

The accuracy peaked at $K = 5$, reaching 73.12% with a Macro F1 score of 0.4172. But we noticed a really cool trade-off! At $K = 18$, the accuracy dropped slightly to 70.94%, but the Macro F1 score shot up to 0.5943. It turns out more clusters let the model represent minority groups instead of letting them get outvoted. Quite neat!

3.2 MLP Classification

We tested our MLP classifier (hidden size = 64, learning rate $lr = 0.01$) across epochs $\in [50, 500]$ to compare standard SGD against our Momentum implementation ($\gamma = 0.9$).

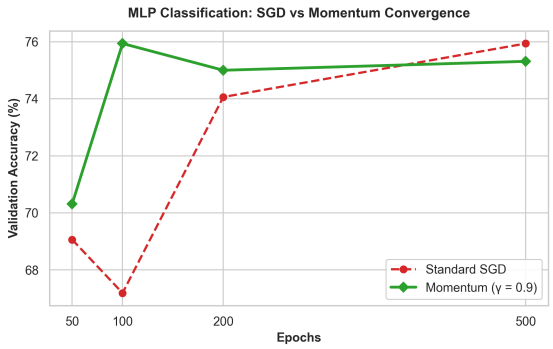

Figure 2: MLP Classification: SGD vs Momentum Validation Accuracy

We hit the jackpot with Momentum! It reached a validation accuracy of 75.94% at just 100 epochs. Standard SGD required a full 500 epochs to reach that same level. Momentum adds inertia to the update step, which accelerates training and dampens oscillations in narrow loss valleys.

3.3 MLP Regression

We tuned the regressor (hidden size = 64, learning rate $lr = 0.001$) on the continuous addiction score task.

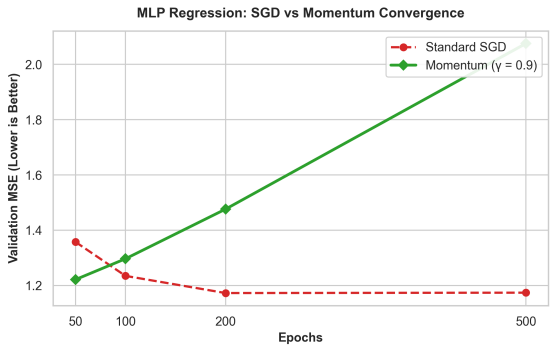

Figure 3: MLP Regression: SGD vs Momentum Validation MSE

Tuning made a massive difference. Standard SGD steadily converged to a validation MSE of 1.1732 at 200 epochs. Momentum was faster at the start, achieving a low MSE of 1.2218 at 50 epochs (compared to SGD's 1.3580). However, at higher epochs, Momentum overshot the mark and diverged (MSE rose to 2.0752). This is a classic gradient descent limitation where too much inertia pushes the model past the local minimum.

3.4 Summary of Results

The table below displays the final performance of our optimized Milestone 2 models.

Task	Model (Parameters)	Performance
Classification	K-Means ($K = 5$)	Acc: 73.12%, F1: 0.4172
Classification	K-Means ($K = 18$)	Acc: 70.94%, F1: 0.5943
Classification	MLP + Momentum (ep=100)	Acc: 75.94%, F1: 0.4556
Regression	MLP + SGD (ep=200)	MSE: 1.1732

Table 1: Summary of final results

4 Extensions

We tried three extra extensions to squeeze more juice out of our pipeline:

- **Momentum Optimizer (Slide 69):** We added inertia to parameter updates:

$$\mathbf{v} \leftarrow \gamma \mathbf{v} + \eta \nabla R(\mathbf{W}) \tag{3}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \mathbf{v} \tag{4}$$

  This damped oscillations and dramatically accelerated classification convergence.

- **Categorical Cross-Entropy from Scratch:** We implemented multi-class cross-entropy and its gradient vectorization from scratch.

- **Deep MLP Architectures:** We tested a deep network configuration of $(D, 64, 32, C)$. Surprisingly, it did not help much and overfit the training set.

5 Discussion and Conclusion

We successfully implemented and tuned all methods. For regression, the MLP achieved a solid 1.1732 MSE. Non-linear mapping is definitely the way to go here. For classification, MLP beat K-Means (75.94% vs 73.12%). Supervised models have a clear edge since they know the target labels. In conclusion, adding momentum was a game-changer for speed, but required careful tuning to avoid overshooting. We look forward to Milestone 3!